

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\interpolation\MicrosphereProjectionInterpolator.java

- Specification

module MicrosphereProjectionInterpolator

type

```
d: int // problem dimension (number of coordinates)
elements: int // number of microsphere surface elements
maxDarkFraction: real // fraction of "dark" facets allowed
darkThreshold: real // illumination threshold
brightnessExponent: real // weighting exponent
sphereRadius: real // microsphere radius (scale)
background: real // fallback value when illumination is too low
sharedSphere: Boolean // whether to reuse the microsphere instance
```

// Samples for interpolation

X: sequence of sequence of real // size $n \times d$ (scattered sites)

Y: sequence of real // size n

n: int

// Query point

p: sequence of real // size d

ex: set of {

```
None,
NullPointerException,
NoDataException,
DimensionMismatchException,
NumbersTooSmallException,
NotFiniteNumberException,
NotPositiveException,
NotStrictlyPositiveException,
OutOfRangeException
```

}

process Construct(d , elements, maxDarkFraction, darkThreshold, brightnessExponent, sphereRadius, background: real, sharedSphere: Boolean) outputs: ex: set

Functional Scenario Form

$d < 1$ && ex =

NumbersTooSmallException ||

elements < $d + 1$ && ex = NumbersTooSmallException ||

```

    maxDarkFraction < 0      && ex = OutOfRangeException ||
darkThreshold > 1          && ex = OutOfRangeException ||
    brightnessExponent <= 0      && ex =
NotStrictlyPositiveException ||
    sphereRadius <= 0          && ex =
NotStrictlyPositiveException ||
finite=0                  && ex = NotFiniteNumberException ||
finite=1                  && ex = None
end_process

```

process Interpolate(X: sequence of sequence of real,
Y: sequence of real) outputs: ex: set

Functional Scenario Form

```

(X = null or Y = null)      && (ex = NullArgumentException)
or
(|X| = 0 or |Y| = 0)        && (ex = NoDataException) or
(|X| ≠ |Y|)                  && (ex =
DimensionMismatchException) or
// Each row non-null and width equals d
(∃ i • i ∈ 0..|X|-1 ∧ (X[i] = null or |X[i]| ≠ d))
&& (ex =
DimensionMismatchException) or
// Configured dimension consistency (if the implementation stores 'd')
(d ≥ 1 ∧ |X| ≥ 1 ∧ |X[0]| ≠ d)      && (ex =
DimensionMismatchException) or
// Need at least d+1 scattered sites to interpolate in d-D
(|X| < d + 1)                      && (ex =
NumberIsTooSmallException) or
// Finite checks over all coordinates and values
(∃ i • i ∈ 0..|X|-1 ∧ (∃ k • k ∈ 0..d-1 ∧ ¬finite(X[i][k]) ∨ ¬finite(Y[i])))
&& (ex =
NotFiniteNumberException) or
(true)                            && (ex = None)
end_process

```

process Value(p: sequence of real) outputs: ex: set

Functional Scenario Form

(p = null) && (ex =
NullPointerException) or

(|p| $\neq$ d) && (ex =
DimensionMismatchException) or

($\exists k \bullet k \in 0..d-1 \wedge \neg \text{finite}(p[k])$) && (ex =
NotFiniteNumberException) or

// Note: typical implementations return 'background' when illumination is too low,
// not an exception. Hence success here covers both normal and fallback usage.

(true) && (ex = None)

end_process

end_module

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *     http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.interpolation;
import org.apache.commons.rng.simple.RandomSource;
import java.util.Random;
import org.apache.commons.rng.sampling.UnitSphereSampler;
import org.apache.commons.math4.legacy.analysis.MultivariateFunction;
```

```
import org.apache.commons.math4.legacy.exception.DimensionMismatchException;
import org.apache.commons.math4.legacy.exception.NoDataException;
import org.apache.commons.math4.legacy.exception.NotPositiveException;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.*;
import java.lang.reflect.*;
import java.util.Arrays;
/**
 * Interpolator that implements the algorithm described in <em>William
 * Dudziak</em>'s <a href="http://www.dudziak.com/microsphere.pdf">MS
 * thesis</a>.
 *
 * @since 3.6
 */
public class MicrosphereProjectionInterpolator implements MultivariateInterpolator {
    /** Brightness exponent. */
    private final double exponent;
    /** Microsphere. */
    private final InterpolatingMicrosphere microsphere;
    /** Whether to share the sphere. */
    private final boolean sharedSphere;
    /** Tolerance value below which no interpolation is necessary. */
    private final double noInterpolationTolerance;
    /**
     * Create a microsphere interpolator.
     *
     * @param dimension Space dimension.
     * @param elements Number of surface elements of the
     *                  microsphere.
     * @param exponent Exponent used in the power law that computes
     *                  the
     * @param maxDarkFraction Maximum fraction of the facets that
     *                  can be
     *                  dark. If the fraction of "non-
     *                  illuminated"
     *                  facets is larger, no estimation of the
     *                  value
     *                  will be performed, and the {@code background}
     *                  value will be returned instead.
     */
}
```

```

    * @param darkThreshold      Value of the illumination below which
a facet
    *                          is considered dark.
    * @param background        Value returned when the
    *                          {@code maxDarkFraction} threshold is
    *                          exceeded.
    * @param sharedSphere      Whether the sphere can be shared among the
    *                          interpolating function instances. If
    *                          {@code true}, the instances will share
the
    *                          the
    *                          same data, and thus will not
be
    *                          thread-safe.
    * @param noInterpolationTolerance When the distance between an interpolated
    *                          point and one of the sample points is
less
    *                          than this value, no interpolation will
be
    *                          performed (the value of the sample will
l be
    *                          returned).
    * @throws org.apache.commons.math4.exception.NotStrictlyPositiveException if
    *
    *          {@code dimension <= 0}
    *
    *          or
    *
    *          {@code elements <= 0}.
    * @throws NotPositiveException
    *          if
    *
    *          {@code exponent < 0}.
    * @throws NotPositiveException
    *          if
    *
    *          {@code darkThreshold < 0}.
    * @throws org.apache.commons.math4.exception.OutOfRangeException if
on
    *
    *          {@code maxDarkFraction}

```

```

*
*   does
*
*   not
*
*   belong
*
*   to
*
*   the
*
*   interval
*
*   {@code [0, 1]}.
*/
public MicrosphereProjectionInterpolator(int dimension, int elements,
double maxDarkFraction, double darkThreshold,
double background, double exponent, boolean sharedSphere, double noI
nterpolationTolerance) {
    this(new InterpolatingMicrosphere(dimension, elements, maxDarkFractio
n, darkThreshold, background,
        UnitSphereSampler.of(RandomSource.MT_64.create(), dimension)), expo
nent, sharedSphere,
        noInterpolationTolerance);
}
/**
 * Create a microsphere interpolator.
 *
 * @param microsphere      Microsphere.
 * @param exponent          Exponent used in the power law that c
omputes
 *                          the weights (distance dimming factor)
of the
 *                          sample data.
 * @param sharedSphere      Whether the sphere can be shared amon
g the
 *                          interpolating function instances. If
 *                          {@code true}, the instances will share
the
 *                          same data, and thus will <em>not</em>
be
 *                          thread-safe.
 * @param noInterpolationTolerance When the distance between an interp
olated

```

```

        * point and one of the sample points is
less
        * than this value, no interpolation will
be
        * performed (the value of the sample wil
l be
        * returned).
        * @throws NotPositiveException if {@code exponent < 0}.
        */
    public MicrosphereProjectionInterpolator(InterpolatingMicrosphere micr
osphere, double exponent,
        boolean sharedSphere, double noInterpolationTolerance) throws NotPos
itiveException {
        if (exponent < 0) {
            throw new NotPositiveException(exponent);
        }
        this.microsphere = microsphere;
        this.exponent = exponent;
        this.sharedSphere = sharedSphere;
        this.noInterpolationTolerance = noInterpolationTolerance;
    }
    /**
     * {@inheritDoc}
     *
     * @throws DimensionMismatchException if the space dimension of the gi
ven
     * samples does not match the space dim
ension
     * of the microsphere.
     */
    @Override
    public MultivariateFunction interpolate(final double[][] xval, final d
ouble[] yval)
        throws DimensionMismatchException, NoDataException, NullArgumentExce
ption {
        if (xval == null || yval == null) {
            throw new NullArgumentException();
        }
        if (xval.length == 0) {
            throw new NoDataException();
        }
        if (xval.length != yval.length) {
            throw new DimensionMismatchException(xval.length, yval.length);
        }
    }

```

```

    if (xval[0] == null) {
        throw new NullPointerException();
    }
    final int dimension = microsphere.getDimension();
    if (dimension != xval[0].length) {
        throw new DimensionMismatchException(xval[0].length, dimension);
    }
    // Microsphere copy.
    final InterpolatingMicrosphere m = sharedSphere ? microsphere : microsphere.copy();
    return new MultivariateFunction() {
        /** {inheritDoc} */
        @Override
        public double value(double[] point) {
            return m.value(point, xval, yval, exponent, noInterpolationTolerance);
        }
    };
}
}
//vt
class MPI_VT {
    private Object newMPI(int d, int elements, double maxDark, double darkThr, double brightExp, double background, boolean shared) {
        final Class<?> cls = MicrosphereProjectionInterpolator.class;
        Object unitSphereGen = null;
        try {
            Class<?> genClz = Class.forName("org.apache.commons.math4.legacy.random.UnitSphereRandomVectorGenerator");
            Constructor<?> gc = null;
            for (Constructor<?> c : genClz.getDeclaredConstructors()) {
                Class<?>[] pt = c.getParameterTypes();
                if (pt.length == 1 && (pt[0] == int.class || pt[0] == Integer.class)) {
                    gc = c;
                    break;
                }
            }
            if (gc != null) {
                gc.setAccessible(true);
                unitSphereGen = gc.newInstance(d);
            }
        } catch (Throwable ignore) {

```

```

unitSphereGen = null;
}
double[][] doublePresets = new double[][] {
    // maxDark, darkThr, brightExp, sphereRad, background
    { clamp01(maxDark), clamp01(darkThr), posOr(brightExp, 2.0), posOr(
1.0, 1.0), background },
    { 0.5, 0.1, 2.0, 1.0, background }, { 0.2, 0.0, 1.5, 1.0, backgroun
d },
    { 0.8, 0.2, 3.0, 1.0, background }, { 0.5, 0.1, 2.0, 1.0, 0.0 }, };
for (Constructor<> ctor : cls.getConstructors()) {
    final Class<>[] pt = ctor.getParameterTypes();
    for (double[] dp : doublePresets) {
        int iInt = 0, iDbl = 0, iBool = 0;
        Object[] args = new Object[pt.length];
        boolean ok = true;
        for (int i = 0; i < pt.length; i++) {
            Class<> t = pt[i];
            if (t == int.class || t == Integer.class) {
                int val = (iInt == 0) ? d : (iInt == 1) ? elements : Math.max(2,
elements);
                args[i] = val;
                iInt++;
            } else if (t == double.class || t == Double.class) {
                double v = (iDbl < dp.length) ? dp[iDbl] : 1.0;
                if (iDbl == 0 || iDbl == 1)
                    v = clamp01(v); // maxDark / darkThr
                if (iDbl == 2 || iDbl == 3)
                    v = strictlyPos(v); // brightness / sphereRadius
                if (Double.isNaN(v) || Double.isInfinite(v))
                    v = 0.0;
                args[i] = v;
                iDbl++;
            } else if (t == boolean.class || t == Boolean.class) {
                args[i] = (iBool == 0) ? Boolean.valueOf(shared) : Boolean.TRUE;
                iBool++;
            } else if (unitSphereGen != null && t.isAssignableFrom(unitSphereG
en.getClass())) {
                args[i] = unitSphereGen;
            } else if (t.isArray()) {
                args[i] = null;
            } else {
                ok = false;
                break;
            }
        }
    }
}

```

```

    }
    if (!ok)
        continue;
    try {
        ctor.setAccessible(true);
        return ctor.newInstance(args);
    } catch (Throwable ignore) {
    }
}
}
}
throw new IllegalStateException("No matching MicrosphereProjectionInt
erpolator constructor found.");
}
private static double clamp01(double v) {
    return v < 0 ? 0 : (v > 1 ? 1 : v);
}
private static double posOr(double v, double fallback) {
    return v > 0 ? v : fallback;
}
private static double strictlyPos(double v) {
    return v > 0 ? v : 1.0;
}
private static Class<?> wrap(Class<?> c) {
    if (!c.isPrimitive())
        return c;
    if (c == int.class)
        return Integer.class;
    if (c == double.class)
        return Double.class;
    if (c == boolean.class)
        return Boolean.class;
    return c;
}
private static boolean canWidenNumber(Class<?> from, Class<?> to) {
    return Number.class.isAssignableFrom(from) && (to == Double.class ||
to == Number.class || to == Object.class);
}
private Object callInterpolate(Object mpi, double[][] X, double[] Y) t
hrows Exception {
    Method m = null;
    for (Method cand : mpi.getClass().getMethods()) {
        if ("interpolate".equals(cand.getName()))
            continue;
        Class<?>[] pt = cand.getParameterTypes();

```

```

        if (pt.length == 2 && pt[0].isArray() && pt[0].getComponentType().isArray() && // double[][]
            pt[1].isArray()) { // double[]
            m = cand;
            break;
        }
    }
    if (m == null)
        throw new IllegalStateException("interpolate(double[][],double[]) not found");
    try {
        return m.invoke(mpi, X, Y);
    } catch (InvocationTargetException ex) {
        Throwable cause = ex.getCause();
        if (cause instanceof RuntimeException)
            throw (RuntimeException) cause;
        if (cause instanceof Error)
            throw (Error) cause;
        throw ex;
    }
}

private double callValue(Object f, double[] p) throws Exception {
    Method m = null;
    for (Method cand : f.getClass().getMethods()) {
        if (!"value".equals(cand.getName()))
            continue;
        Class<?>[] pt = cand.getParameterTypes();
        if (pt.length == 1 && pt[0].isArray()) {
            m = cand;
            break;
        }
    }
    if (m == null)
        throw new IllegalStateException("value(double[]) not found on returned function");
    try {
        Object r = m.invoke(f, new Object[] { p });
        return ((Number) r).doubleValue();
    } catch (InvocationTargetException ex) {
        Throwable cause = ex.getCause();
        if (cause instanceof RuntimeException)
            throw (RuntimeException) cause;
        if (cause instanceof Error)
            throw (Error) cause;
    }
}

```

```

        throw ex;
    }
}

private Object newMPI_withNegativeExponent(int d, int elements, double
maxDark, double darkThr, double background,
    boolean shared) {
    final Class<?> cls = MicrosphereProjectionInterpolator.class;
    final double SAFE_BRIGHT = 2.0; // >0
    final double SAFE_RADIUS = 1.0; // >0
    final double SAFE_MAXD = Math.min(Math.max(maxDark, 0.0), 1.0);
    final double SAFE_THR = Math.min(Math.max(darkThr, 0.0), 1.0);
    final double SAFE_BG = (Double.isFinite(background) ? background : 0.
0);
    Object unitSphereGen = null;
    try {
        Class<?> genClz = Class.forName("org.apache.commons.math4.legacy.random.UnitSphereRandomVectorGenerator");
        for (java.lang.reflect.Constructor<?> c : genClz.getDeclaredConstructors()) {
            Class<?>[] pt = c.getParameterTypes();
            if (pt.length == 1 && (pt[0] == int.class || pt[0] == Integer.class)) {
                c.setAccessible(true);
                unitSphereGen = c.newInstance(d);
                break;
            }
        }
    } catch (Throwable ignore) {
        unitSphereGen = null;
    }
    ctorLoop: for (java.lang.reflect.Constructor<?> ctor : cls.getConstructors()) {
        Class<?>[] pt = ctor.getParameterTypes();
        Object[] base = new Object[pt.length];
        int iInt = 0, iDbl = 0, iBool = 0;
        boolean ok = true;
        for (int i = 0; i < pt.length; i++) {
            Class<?> t = pt[i];
            if (t == int.class || t == Integer.class) {
                base[i] = (iInt == 0 ? d : Math.max(elements, d + 1));
                iInt++;
            } else if (t == double.class || t == Double.class) {
                double v;
                if (iDbl == 0)

```

```

        v = SAFE_MAXD;
    else if (iDbl == 1)
        v = SAFE_THR;
    else if (iDbl == 2)
        v = SAFE_BRIGHT;
    else if (iDbl == 3)
        v = SAFE_RADIUS;
    else
        v = SAFE_BG;
    base[i] = v;
    iDbl++;
} else if (t == boolean.class || t == Boolean.class) {
    base[i] = Boolean.valueOf(shared);
    iBool++;
} else if (unitSphereGen != null && t.isAssignableFrom(unitSphereGe
n.getClass())) {
    base[i] = unitSphereGen;
} else {
    ok = false;
    break;
}
}
}
if (!ok)
    continue;
for (int j = 0, dblIndex = 0; j < pt.length; j++) {
    if (pt[j] == double.class || pt[j] == Double.class) {
        Object[] args = base.clone();
        args[j] = -1.0;
        try {
            ctor.setAccessible(true);
            ctor.newInstance(args);
        } catch (java.lang.reflect.InvocationTargetException ite) {
            Throwable c = ite.getCause();
            if (c instanceof NotPositiveException) {
                throw (NotPositiveException) c;
            }
        } catch (Throwable ignore) {
        }
        dblIndex++;
    }
}
}
}
throw new IllegalStateException("No constructor exposed a negative-
exponent check.");

```

```

}
@FunctionalInterface
private interface Throwing {
    void run() throws Exception;
}
private static void headline(String name) {
    System.out.println("\n==== " + name + " ====");
}
private static void expectThrow(String name, Throwing r, Class<?> expected) {
    System.out.println("---- " + name + " ----");
    try {
        r.run();
        System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
    } catch (Throwable t) {
        Throwable c = (t instanceof InvocationTargetException && t.getCause() != null) ? t.getCause() : t;
        System.out.println("threw: " + c.getClass().getName() + " - " + c.getMessage());
        System.out.printf("[%s] %s threw expected\n", c.getClass().equals(expected) ? "PASS" : "FAIL", name);
    }
}
private static String fmt(double v) {
    if (Double.isNaN(v))
        return "NaN";
    if (v == Double.POSITIVE_INFINITY)
        return "+Inf";
    if (v == Double.NEGATIVE_INFINITY)
        return "-Inf";
    if (v == 0.0)
        return (Double.doubleToRawLongBits(v) >>> 63) == 1 ? "-0.0" : "0.0";
    return String.format(java.util.Locale.ROOT, "%.17g", v);
}
private static void checkClose(String name, double exp, double got, double relTol) {
    boolean ok;
    if (Double.isNaN(exp) || Double.isNaN(got))
        ok = (Double.isNaN(exp) && Double.isNaN(got));
    else if (Double.isInfinite(exp) || Double.isInfinite(got))
        ok = (Double.doubleToRawLongBits(exp) == Double.doubleToRawLongBits(got));
    else {

```

```

        double tol = Math.max(1.0, Math.max(Math.abs(exp), Math.abs(got))) *
relTol;
        ok = Math.abs(exp - got) <= tol;
    }
    System.out.printf("[%s] %s expected=%s actual=%s (relTol=%.1e)%n", o
k ? "PASS" : "FAIL", name, fmt(exp),
        fmt(got), relTol);
    }
    private static double[][] grid2D(double x0, double y0, double step, in
t nx, int ny) {
        double[][] X = new double[nx * ny][2];
        int k = 0;
        for (int i = 0; i < nx; i++)
            for (int j = 0; j < ny; j++) {
                X[k][0] = x0 + i * step;
                X[k][1] = y0 + j * step;
                k++;
            }
        return X;
    }
    private static double[] planeZ(double[][] X, double a, double b) { //
z=a*x + b*y
        double[] Y = new double[X.length];
        for (int i = 0; i < X.length; i++)
            Y[i] = a * X[i][0] + b * X[i][1];
        return Y;
    }
    private static double[] lin(double a, double h, int n) {
        double[] v = new double[n];
        for (int i = 0; i < n; i++)
            v[i] = a + i * h;
        return v;
    }
    private static double[][] zip(double[] x, double[] y, double[] z) {
        int n = x.length;
        double[][] X = new double[n][3];
        for (int i = 0; i < n; i++) {
            X[i][0] = x[i];
            X[i][1] = y[i];
            X[i][2] = z[i];
        }
        return X;
    }
    public void Vtest() throws Exception {

```

```

final int dOK = 2, e1OK = 64;
final double maxDark = 0.5, darkThr = 0.1, brightExpOK = 2.0, bgOK =
0.0;
final boolean shared = true;
// VT01: d < 1 -> NumberIsTooSmallException
expectThrow("VT01 ctor d<1", () -> newMPI(0, e1OK, maxDark, darkThr,
brightExpOK, bgOK, shared),
    NumberIsTooSmallException.class);
// VT02: elements < 1 或 < d+1 -> NumberIsTooSmallException
expectThrow("VT02 ctor elements<d+1", () -> newMPI(dOK, dOK, maxDark,
darkThr, brightExpOK, bgOK, shared),
    NumberIsTooSmallException.class);
// VT03: maxDarkFraction < 0 -> OutOfRangeException
expectThrow("VT03 ctor maxDarkFraction<0", () -> newMPI(dOK, e1OK, -
0.01, darkThr, brightExpOK, bgOK, shared),
    OutOfRangeException.class);
// VT04: maxDarkFraction > 1 -> OutOfRangeException
expectThrow("VT04 ctor maxDarkFraction>1", () -> newMPI(dOK, e1OK, 1.
01, darkThr, brightExpOK, bgOK, shared),
    OutOfRangeException.class);
// VT05: darkThreshold < 0 -> OutOfRangeException
expectThrow("VT05 ctor darkThreshold<0", () -> newMPI(dOK, e1OK, maxD
ark, -1e-3, brightExpOK, bgOK, shared),
    OutOfRangeException.class);
// VT06: darkThreshold > 1 -> OutOfRangeException
expectThrow("VT06 ctor darkThreshold>1", () -> newMPI(dOK, e1OK, maxD
ark, 1.1, brightExpOK, bgOK, shared),
    OutOfRangeException.class);
// VT07: brightnessExponent <= 0 -> NotStrictlyPositiveException
expectThrow("VT07 ctor brightnessExponent<=0", () -> newMPI(dOK, e1OK
, maxDark, darkThr, 0.0, bgOK, shared),
    NotStrictlyPositiveException.class);
// VT08:
expectThrow("VT08 ctor background=NaN",
    () -> newMPI(dOK, e1OK, maxDark, darkThr, brightExpOK, Double.NaN,
shared),
    NotFiniteNumberException.class);
final Object MPI_OK = newMPI(dOK, e1OK, maxDark, darkThr, brightExpOK
, bgOK, shared);
// VT09: X = null -> NullArgumentException
expectThrow("VT09 interpolate X=null", () -> callInterpolate(MPI_OK,
null, new double[] { 1 })),
    NullArgumentException.class);
// VT10: Y = null -> NullArgumentException

```

```

    expectThrow("VT10 interpolate Y=null", () -> callInterpolate(MPI_OK,
new double[][] { { 0, 0 } }, null),
    NullArgumentException.class);
    // VT11: |X|=0 / |Y|=0 -> NoDataException
    expectThrow("VT11 empty samples", () -> callInterpolate(MPI_OK, new d
ouble[][] {}, new double[] {}),
    NoDataException.class);
    // VT12: |X| ≠ |Y| -> DimensionMismatchException
    expectThrow("VT12 size mismatch",
    () -> callInterpolate(MPI_OK, new double[][] { { 0, 0 }, { 1, 1 } }
, new double[] { 1.0 }},
    DimensionMismatchException.class);
    // VT13:
    expectThrow("VT13 row width!=d",
    () -> callInterpolate(MPI_OK, new double[][] { { 0, 0, 0 } }, new d
ouble[] { 0.0 }},
    DimensionMismatchException.class);
    // VT14:
    expectThrow("VT14 n<d+1",
    () -> callInterpolate(MPI_OK, new double[][] { { 0, 0 }, { 1, 1 } }
, new double[] { 0, 1 }},
    NumberIsTooSmallException.class);
    // VT15:
    expectThrow(
    "VT15 X has NaN", () -> callInterpolate(MPI_OK,
    new double[][] { { 0, 0 }, { Double.NaN, 1 }, { 1, 2 } }, new dou
ble[] { 0, 1, 2 }},
    NotFiniteNumberException.class);
    // VT16:
    expectThrow("VT16 Y has +Inf", () -> callInterpolate(MPI_OK, new doub
le[][] { { 0, 0 }, { 1, 1 }, { 2, 2 } },
    new double[] { 0, Double.POSITIVE_INFINITY, 2 }}, NotFiniteNumberEx
ception.class);
    // VT17:
    headline("VT17 success 2D plane");
    {
    double[][] X = grid2D(0, 0, 1.0, 3, 3); // 9 点 >= d+1
    double[] Y = planeZ(X, 2.0, 3.0);
    Object f = callInterpolate(MPI_OK, X, Y);
    double got = callValue(f, new double[] { 0.5, 0.25 });
    double exp = 2.0 * 0.5 + 3.0 * 0.25; // 1.75
    checkClose("VT17 value", exp, got, 1e-1);
    }
    // VT18

```

```

    expectThrow("VT18 ctor exponent<0", () -> newMPI_withNegativeExponent
(2, 64, 0.5, 0.1, 0.0, true),
    NotPositiveException.class);
    // VT19
    headline("VT19 interpolate with X[0]=null");
    {
        final Object MPI_OK2 = newMPI(2, 64, 0.5, 0.1, 2.0, 0.0, true);
        double[][] X = new double[][] { null, new double[] { 0, 0 }, new dou
ble[] { 1, 1 } };
        double[] Y = new double[] { 0.0, 0.0, 2.0 };
        expectThrow("VT22 X[0]==null", () -> callInterpolate(MPI_OK2, X, Y),
        NullArgumentException.class);
    }
    // VT20
    headline("VT23 sharedSphere=false -> microsphere.copy()");
    {
        final Object MPI_COPY = newMPI(2, 64, 0.5, 0.1, 2.0, 0.0, false);
        double[][] X = grid2D(0, 0, 1.0, 3, 3);
        double[] Y = planeZ(X, 1.0, 2.0); // z = x + 2y
        Object f = callInterpolate(MPI_COPY, X, Y);
        double val = callValue(f, new double[] { 0.5, 0.25 });
        checkClose("VT23 value", 1.0, val, 2e-1);
    }
    System.out.println("\n[Done] MicrosphereProjectionInterpolator VT01-
VT20 executed.");
}
}
//ft
class MPI_FT{
    private static void headline(String t){ System.out.println("\n===
" + t + " ==="); }
    private static String fmt(double v){
        if (Double.isNaN(v)) return "NaN";
        if (v == Double.POSITIVE_INFINITY) return "+Inf";
        if (v == Double.NEGATIVE_INFINITY) return "-Inf";
        if (v == 0.0) return (Double.doubleToRawLongBits(v)>>>63)==1 ?
"-0.0" : "0.0";
        return String.format(java.util.Locale.ROOT, "%.7g", v);
    }
    private Object newSphere(int d, Random R) {
        try {
            Class<?> sCls = Class.forName("org.apache.commons.math4.leg
acy.analysis.interpolation.InterpolatingMicrosphere");
            Object unit = null;

```

```

        try {
            Class<?> genClz = Class.forName("org.apache.commons.math4.legacy.random.UnitSphereRandomVectorGenerator");
            for (Constructor<?> c : genClz.getDeclaredConstructors()) {
                Class<?>[] pt = c.getParameterTypes();
                if (pt.length == 1 && (pt[0]==int.class || pt[0]==Integer.class)) {
                    c.setAccessible(true);
                    unit = c.newInstance(d);
                    break;
                }
            }
        } catch (Throwable ignore) {}
        for (Constructor<?> c : sCls.getConstructors()) {
            Class<?>[] pt = c.getParameterTypes();
            Object[] args = new Object[pt.length];
            boolean ok = true;
            int iInt=0, iDbl=0, iBool=0;
            for (int i=0;i<pt.length;i++){
                Class<?> t = pt[i];
                if (t==int.class || t==Integer.class) {
                    int v = (iInt==0) ? d : Math.max(d+1, 32 + R.nextInt(96));

                    args[i]=v; iInt++;
                } else if (t==double.class || t==Double.class) {
                    double v;
                    if (iDbl==0) v = 1.0;
                    else if (iDbl==1) v = 2.0;
                    else v = 0.1 + R.nextDouble();
                    if (!Double.isFinite(v) || v<=0) v = 1.0;
                    args[i]=v; iDbl++;
                } else if (t==boolean.class || t==Boolean.class) {
                    args[i] = Boolean.TRUE; iBool++;
                } else if (unit!=null && t.isAssignableFrom(unit.getClass())) {
                    args[i] = unit;
                } else if (t.isArray()) {
                    args[i] = Array.newInstance(t.getComponentType(), 0);
                } else {
                    ok=false; break;
                }
            }
        }
    }

```

```

        if (!ok) continue;
        try { c.setAccessible(true); return c.newInstance(args)
; }

        catch (Throwable ignore) {}
    }
} catch (Throwable e) { /* ignore */ }
throw new IllegalStateException("Unable to construct Interpolat
ingMicrosphere");
}
private Object newMPI(int d, int elements, double maxDark, double d
arkThr,

                        double brightExp, double sphereRadius,
                        double background, boolean shared, Random R)
{
    final Class<?> cls = MicrosphereProjectionInterpolator.class;
    final double[][] presets = new double[][]{
        { clamp01(maxDark), clamp01(darkThr), pos(brightExp,2.0), p
os(sphereRadius,1.0), finite(background,0.0) },
        { 0.5, 0.1, 2.0, 1.0, finite(background,0.0) },
        { 0.2, 0.0, 1.5, 1.0, finite(background,0.0) },
        { 0.8, 0.2, 3.0, 1.0, 0.0 }
    };
    Object sphereCandidate = null;
    try { sphereCandidate = newSphere(d, R); } catch (Throwable ign
ore) {}
    for (Constructor<?> ctor : cls.getConstructors()) {
        Class<?>[] pt = ctor.getParameterTypes();
        if (sphereCandidate != null && pt.length==4 &&
            pt[0].isAssignableFrom(sphereCandidate.getClass()) &&
            (pt[1]==double.class || pt[1]==Double.class) &&
            (pt[2]==boolean.class || pt[2]==Boolean.class) &&
            (pt[3]==double.class || pt[3]==Double.class)) {
            try {
                ctor.setAccessible(true);
                return ctor.newInstance(sphereCandidate, 2.0, Boole
an.valueOf(shared), 1e-6);
            } catch (Throwable ignore) {}
        }
        for (double[] dp : presets) {
            Object[] args = new Object[pt.length];
            int iInt=0, iDbl=0, iBool=0;
            boolean ok = true;
            for (int i=0;i<pt.length;i++){
                Class<?> t = pt[i];

```

```

        if (t==int.class || t==Integer.class) {
            int v = (iInt==0) ? Math.max(1,d) : Math.max(d+
1, elements);

            args[i]=v; iInt++;
        } else if (t==double.class || t==Double.class) {
            double v = (iDbl < dp.length) ? dp[iDbl] : 1.0;
            if (iDbl==0 || iDbl==1) v = clamp01(v);
            if (iDbl==2 || iDbl==3) v = pos(v, 1.0);
            if (!Double.isFinite(v)) v = 0.0;
            args[i]=v; iDbl++;
        } else if (t==boolean.class || t==Boolean.class) {
            args[i] = Boolean.valueOf(shared); iBool++;
        } else if (sphereCandidate!=null && t.isAssignableFrom(sphereCandidate.getClass())) {
            args[i] = sphereCandidate;
        } else {
            ok=false; break;
        }
    }
    if (!ok) continue;
    try { ctor.setAccessible(true); return ctor.newInstance
(args); }

    catch (Throwable ignore) {}
}
}
throw new IllegalStateException("No matching MicrosphereProject
ionInterpolator constructor found.");
}
private static double[][] genX(Random R, int n, int d) {
    double[][] X = new double[n][d];
    double[] base = new double[d];
    for (int k=0;k<d;k++) base[k] = R.nextGaussian() * 2.0;
    for (int i=0;i<n;i++){
        for (int k=0;k<d;k++){
            double v = base[k] + (R.nextGaussian()* (1.0 + 2.0*R.ne
xtDouble()));
            if (R.nextDouble()<0.02) v = Double.NaN;
            else if (R.nextDouble()<0.02) v = Double.POSITIVE_INFIN
ITY;
            else if (R.nextDouble()<0.02) v = Double.NEGATIVE_INFIN
ITY;
            X[i][k] = v;
        }
    }
}

```

```

        if (R.nextDouble() < 0.1 && i>0) X[i] = Arrays.copyOf(X[i-1], d);
    }
    if (R.nextDouble()<0.2) {
        for (int i=0;i<n;i++){
            int j = R.nextInt(n);
            double[] tmp = X[i]; X[i]=X[j]; X[j]=tmp;
        }
    }
    return X;
}

private static double[] genY(Random R, double[][] X) {
    int n = X.length, d = X[0].length;
    double[] w = new double[d];
    for (int k=0;k<d;k++) w[k] = R.nextGaussian();
    double b = R.nextGaussian();
    double[] Y = new double[n];
    for (int i=0;i<n;i++){
        double s = b;
        for (int k=0;k<d;k++){
            double v = X[i][k];
            if (!Double.isFinite(v)) { s = Double.NaN; break; }
            s += w[k]*v;
        }
        if (R.nextDouble()<0.3) s += R.nextGaussian()*0.05;
        if (R.nextDouble()<0.02) s = Double.NaN;
        else if (R.nextDouble()<0.02) s = Double.POSITIVE_INFINITY;
        else if (R.nextDouble()<0.02) s = Double.NEGATIVE_INFINITY;
        Y[i]=s;
    }
    return Y;
}

private static double[] genP(Random R, int d){
    double[] p = new double[d];
    for (int k=0;k<d;k++){
        double v = R.nextGaussian()*2.0;
        if (R.nextDouble()<0.02) v = Double.NaN;
        p[k]=v;
    }
    return p;
}

public void Ftest() {
    final Random R = new Random(20251028L);
    for (int id=1; id<=20; id++){

```

```

        headline(String.format("FT%02d", id));
        try {
            int d = 1 + R.nextInt(5);
            int elements = Math.max(d+1, 16 + R.nextInt(128));
            boolean shared = R.nextBoolean();
            double maxDark = R.nextDouble();
            double darkThr = R.nextDouble(); // [0,1)
            double brightExp = Math.pow(10, R.nextDouble()*2) * (R.
nextBoolean()?1:1); // >0
            double radius = 0.5 + R.nextDouble()*2.0; // >0
            double background = (R.nextDouble()<0.05) ? Double.NaN
: (R.nextGaussian());
            System.out.printf("cfg: d=%d elements=%d shared=%s maxD
ark=%.3g darkThr=%.3g exp=%.3g radius=%.3g bg=%s\n",
                d, elements, shared, maxDark, darkThr, brightEx
p, radius, fmt(background));
            Object mpi = newMPI(d, elements, maxDark, darkThr, brig
htExp, radius, background, shared, R);
            int n = R.nextInt(81); // 0..80
            if (R.nextDouble()<0.15) n = R.nextInt(3);
            double[][] X = (n==0)? new double[0][Math.max(1,d)] : g
enX(R, n, d);
            double[] Y = (n==0)? new double[0] : genY(R, X);
            Object f;
            try {
                f = callInterpolate(mpi, X, Y);
                System.out.printf("interpolate OK (n=%d,d=%d)%n", n
, d);
            } catch (Throwable t) {
                System.out.printf("interpolate EX (%s): %s\n",
                    t.getClass().getSimpleName(), t.getMessage(
));
                continue;
            }
            double[] p = genP(R, d);
            try {
                double val = callValue(f, p);
                System.out.printf("value(p) OK -> %s\n", fmt(val));
            } catch (Throwable t) {
                System.out.printf("value EX (%s): %s\n",
                    t.getClass().getSimpleName(), t.getMessage(
));
            }
        } catch (Throwable t){

```

```

        System.out.printf("FT%02d FATAL (%s): %s\n", id, t.getClass().getSimpleName(), t.getMessage());
    }
}

System.out.println("\n[Done] MicrosphereProjectionInterpolator
FT01-FT20 fuzzing executed.");
}

private Object callInterpolate(Object mpi, double[][] X, double[] Y
) throws Exception {
    Method m = null;
    for (Method cand : mpi.getClass().getMethods()) {
        if (!"interpolate".equals(cand.getName())) continue;
        Class<?>[] pt = cand.getParameterTypes();
        if (pt.length==2 && pt[0].isArray() && pt[0].getComponentType().isArray() && pt[1].isArray()){
            m = cand; break;
        }
    }
    if (m==null) throw new IllegalStateException("interpolate(double[][],double[]) not found");
    try { return m.invoke(mpi, X, Y); }
    catch (InvocationTargetException ite){
        Throwable c = ite.getCause();
        if (c instanceof RuntimeException) throw (RuntimeException)c;

        if (c instanceof Error) throw (Error)c;
        throw ite;
    }
}

private double callValue(Object f, double[] p) throws Exception {
    Method m = null;
    for (Method cand : f.getClass().getMethods()) {
        if (!"value".equals(cand.getName())) continue;
        Class<?>[] pt = cand.getParameterTypes();
        if (pt.length==1 && pt[0].isArray()) { m = cand; break; }
    }
    if (m==null) throw new IllegalStateException("value(double[]) not found");
    try { Object r = m.invoke(f, new Object[]{ p }); return ((Number)r).doubleValue(); }
    catch (InvocationTargetException ite){
        Throwable c = ite.getCause();
        if (c instanceof RuntimeException) throw (RuntimeException)c;

```

```

        if (c instanceof Error) throw (Error)c;
        throw ite;
    }
}
private static double clamp01(double v){ return v<0?0:(v>1?1:v); }
private static double pos(double v, double fb){ return v>0? v : fb;
}
private static double finite(double v, double fb){ return Double.is
Finite(v)? v : fb; }
}
//Z3
class MPI_Z3 {
    @FunctionalInterface private interface Throwing { void run() throws
Exception; }
    private static void headline(String t){ System.out.println("\n===
" + t + " ==="); }
    private static void expectThrow(String name, Throwing r, Class<?> e
xpected){
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expe
cted.getSimpleName());
        } catch (Throwable t) {
            Throwable c = (t instanceof InvocationTargetException && t.
getCause()!=null) ? t.getCause() : t;
            System.out.println("threw: " + c.getClass().getName() + " -
" + c.getMessage());
            System.out.printf("[%s] %s threw expected\n",
                c.getClass().equals(expected) ? "PASS" : "FAIL", na
me);
        }
    }
    private static String fmt(double v){
        if (Double.isNaN(v)) return "NaN";
        if (v == Double.POSITIVE_INFINITY) return "+Inf";
        if (v == Double.NEGATIVE_INFINITY) return "-Inf";
        if (v == 0.0) return (Double.doubleToRawLongBits(v)>>>63)==1 ?
"-0.0" : "0.0";
        return String.format(java.util.Locale.ROOT, "%.17g", v);
    }
    private Object newMPI_withSphere(Object sphere, double exponent, bo
olean shared, double tol) throws Exception {
        Class<?> cls = MicrosphereProjectionInterpolator.class;

```

```

        for (Constructor<?> ctor : cls.getConstructors()) {
            Class<?>[] pt = ctor.getParameterTypes();
            if (pt.length==4 &&
                pt[0].isAssignableFrom(sphere.getClass()) &&
                (pt[1]==double.class || pt[1]==Double.class) &&
                (pt[2]==boolean.class || pt[2]==Boolean.class) &&
                (pt[3]==double.class || pt[3]==Double.class)) {
                ctor.setAccessible(true);
                return ctor.newInstance(sphere, exponent, shared, tol);
            }
        }
        throw new IllegalStateException("Sphere overload ctor not found");
    }
    private Object newSphere(int d) {
        try {
            Class<?> sCls = Class.forName("org.apache.commons.math4.legacy.analysis.interpolation.InterpolatingMicrosphere");
            Object unitSphereGen = null;
            try {
                Class<?> genClz = Class.forName("org.apache.commons.math4.legacy.random.UnitSphereRandomVectorGenerator");
                for (Constructor<?> c : genClz.getDeclaredConstructors()) {
                    Class<?>[] pt = c.getParameterTypes();
                    if (pt.length == 1 && (pt[0] == int.class || pt[0] == Integer.class)) {
                        c.setAccessible(true);
                        unitSphereGen = c.newInstance(d);
                        break;
                    }
                }
            } catch (Throwable ignore) {}
            for (Constructor<?> ctor : sCls.getConstructors()) {
                Class<?>[] pt = ctor.getParameterTypes();
                Object[] args = new Object[pt.length];
                boolean ok = true;
                int iInt=0, iDbl=0, iBool=0;
                for (int i=0;i<pt.length;i++){
                    Class<?> t = pt[i];
                    if (t==int.class || t==Integer.class){
                        args[i] = (iInt==0 ? d : Math.max(64, d+1));
                        iInt++;
                    } else if (t==double.class || t==Double.class){

```

```

        double v = (iDbl==0 ? 1.0 : (iDbl==1 ? 2.0 : 0.
5));

        if (!Double.isFinite(v) || v<=0) v = 1.0;
        args[i]=v; iDbl++;
    } else if (t==boolean.class || t==Boolean.class){
        args[i] = Boolean.TRUE; iBool++;
    } else if (unitSphereGen != null && t.isAssignableF
rom(unitSphereGen.getClass())){
        args[i] = unitSphereGen;
    } else if (t.isArray()) {
        args[i] = Array.newInstance(t.getComponentType(
), 0);
    } else { ok=false; break; }
    }
    if (!ok) continue;
    try { ctor.setAccessible(true); return ctor.newInstance
(args); }

    catch (Throwable ignore) {}
    }
    } catch (Throwable e) { /* ignore */ }
    throw new IllegalStateException("Unable to construct Interplat
ingMicrosphere via reflection");
    }
    private Object newMPI_auto(int d, int elements, String injKind, boo
lean shared) {
        final Class<?> cls = MicrosphereProjectionInterpolator.class;
        Object sphere = null;
        try { sphere = newSphere(d); } catch (Throwable ignore) {}
        final double[][] presets = new double[][]{
            { 0.25, 0.2, 2.0, 1.0, 0.0 },
            { 0.5, 0.1, 2.5, 1.0, 0.0 },
            { 0.8, 0.3, 3.0, 1.0, 0.0 }
        };
        for (Constructor<?> ctor : cls.getConstructors()) {
            Class<?>[] pt = ctor.getParameterTypes();
            if (sphere != null && pt.length==4 &&
                pt[0].isAssignableFrom(sphere.getClass()) &&
                (pt[1]==double.class || pt[1]==Double.class) &&
                (pt[2]==boolean.class || pt[2]==Boolean.class) &&
                (pt[3]==double.class || pt[3]==Double.class)) {
                try {
                    double exp = injKind.equals("BRIGHT-
LE0") ? 0.0 : 2.0;
                    if (injKind.equals("BRIGHT-LT0")) exp = -1.0;

```

```

        double tol = 1e-6;
        ctor.setAccessible(true);
        return ctor.newInstance(sphere, exp, shared, tol);
    } catch (Throwable ignore) {}
}
for (double[] dp : presets) {
    Object[] args = new Object[pt.length];
    int iInt=0, iDbl=0, iBool=0;
    boolean ok = true;
    for (int i=0;i<pt.length;i++){
        Class<?> t = pt[i];
        if (t==int.class || t==Integer.class){
            args[i] = (iInt==0 ? d : Math.max(d+1, elements
));
            iInt++;
        } else if (t==double.class || t==Double.class){
            double v;
            v = (iDbl < dp.length) ? dp[iDbl] : 1.0;
            if ("MAXDARK-
LOW".equals(injKind) && iDbl==0) v = -0.1;
            if ("MAXDARK-
HIGH".equals(injKind) && iDbl==0) v = 1.5;
            if ("DARKTHR-
LOW".equals(injKind) && iDbl==1) v = -0.1;
            if ("DARKTHR-
HIGH".equals(injKind) && iDbl==1) v = 1.2;
            if ("BRIGHT-
LE0".equals(injKind) && iDbl==2) v = 0.0;
            if ("RADIUS-
LE0".equals(injKind) && iDbl==3) v = 0.0;
            if ("BG-
NAN".equals(injKind) && iDbl>=4) v = Double.NaN;
            args[i]=v; iDbl++;
        } else if (t==boolean.class || t==Boolean.class){
            args[i] = Boolean.valueOf(shared); iBool++;
        } else if (sphere != null && t.isAssignableFrom(sph
ere.getClass())){
            args[i] = sphere;
        } else {
            ok=false; break;
        }
    }
    if (!ok) continue;

```

```

        try { ctor.setAccessible(true); return ctor.newInstance
(args); }

        catch (Throwable ignore) {}

    }

    }

    throw new IllegalStateException("No matching MicrosphereProject
ionInterpolator constructor found.");
}

private Object callInterpolate(Object mpi, double[][] X, double[] Y
) throws Exception {
    Method m = null;
    for (Method cand : mpi.getClass().getMethods()) {
        if (!"interpolate".equals(cand.getName())) continue;
        Class<?>[] pt = cand.getParameterTypes();
        if (pt.length==2 && pt[0].isArray() && pt[0].getComponentTy
pe().isArray() && pt[1].isArray()){
            m = cand; break;
        }
    }
    if (m==null) throw new IllegalStateException("interpolate(doubl
e[][],double[]) not found");
    try { return m.invoke(mpi, X, Y); }
    catch (InvocationTargetException ite){
        Throwable c = ite.getCause();
        if (c instanceof RuntimeException) throw (RuntimeException)
c;

        if (c instanceof Error) throw (Error)c;
        throw ite;
    }
}

private double callValue(Object f, double[] p) throws Exception {
    Method m = null;
    for (Method cand : f.getClass().getMethods()) {
        if (!"value".equals(cand.getName())) continue;
        Class<?>[] pt = cand.getParameterTypes();
        if (pt.length==1 && pt[0].isArray()) { m = cand; break; }
    }
    if (m==null) throw new IllegalStateException("value(double[]) n
ot found");
    try { Object r = m.invoke(f, new Object[]{ p }); return ((Numbe
r)r).doubleValue(); }
    catch (InvocationTargetException ite){
        Throwable c = ite.getCause();

```

```

        if (c instanceof RuntimeException) throw (RuntimeException)
c;

        if (c instanceof Error) throw (Error)c;
        throw ite;
    }
}

private static double[][] X_rows(double[]... rows){ return rows; }
public void Z3test() throws Exception {
    // Z301:  $d < 1 \rightarrow \text{NumberIsTooSmall}$ 
    expectThrow("Z301 ctor d<1",
        () -> newMPI_auto(0, 1, "OK", true),
        NumberIsTooSmallException.class);
    // Z302:  $\text{elements} < 1 \rightarrow \text{NumberIsTooSmall}$ 
    expectThrow("Z302 ctor elements<1",
        () -> newMPI_auto(2, 0, "OK", true),
        NumberIsTooSmallException.class);
    // Z303:  $\text{elements} < d+1 \rightarrow \text{NumberIsTooSmall}$ 
    expectThrow("Z303 ctor elements<d+1",
        () -> newMPI_auto(3, 3, "OK", true),
        NumberIsTooSmallException.class);
    // Z304:  $\text{maxDarkFraction}$  out of  $[0,1] \rightarrow \text{OutOfRangeException}$ 
    expectThrow("Z304 ctor maxDarkFraction>1",
        () -> newMPI_auto(2, 6, "MAXDARK-HIGH", true),
        OutOfRangeException.class);
    // Z305:  $\text{darkThreshold}$  out of  $[0,1] \rightarrow \text{OutOfRangeException}$ 
    expectThrow("Z305 ctor darkThreshold<0",
        () -> newMPI_auto(2, 6, "DARKTHR-LOW", true),
        OutOfRangeException.class);
    // Z306:  $\text{brightnessExponent} \leq 0 \rightarrow \text{NotStrictlyPositive}$ 
    expectThrow("Z306 ctor brightnessExponent<=0",
        () -> {
            try {
                Object sphere = newSphere(2);
                newMPI_withSphere(sphere, 0.0, true, 1e-6);
            } catch (Throwable ignore) {
                newMPI_auto(2, 6, "BRIGHT-LE0", true);
            }
        },
        NotStrictlyPositiveException.class);
    // Z307:  $\text{sphereRadius} \leq 0 \rightarrow \text{NotStrictlyPositive}$ 
    expectThrow("Z307 ctor radius<=0",
        () -> newMPI_auto(2, 6, "RADIUS-LE0", true),
        NotStrictlyPositiveException.class);
    // Z308:  $\text{background}$  not finite  $\rightarrow \text{NotFiniteNumber}$ 

```

```

expectThrow("Z308 ctor background=NaN",
    () -> newMPI_auto(2, 6, "BG-NAN", true),
    NotFiniteNumberException.class);
// Z309: Construct: OK → None
headline("Z309 ctor OK");
final Object MPI_OK = newMPI_auto(3, 8, "OK", true);
System.out.println("Z309 constructed OK");
// Z310: Interpolate: X=null or Y=null → NullArgument
expectThrow("Z310 interpolate X=null",
    () -> callInterpolate(MPI_OK, null, new double[][]{1.0}),
    NullArgumentException.class);
// Z311: |X|=0 or |Y|=0 → NoData
expectThrow("Z311 interpolate empty",
    () -> callInterpolate(MPI_OK, new double[][]{}, new double[][]{}),
    NoDataException.class);
// Z312: |X| ≠ |Y| → DimensionMismatch
expectThrow("Z312 size mismatch",
    () -> callInterpolate(MPI_OK, X_rows(new double[]{}{0,0},
new double[]{}{1,1}, new double[]{}{2,2}), new double[]{}{1.0, 2.0}),
    DimensionMismatchException.class);
// Z313: ∃ row null or width≠d → DimensionMismatch
expectThrow("Z313 bad row (null/width!=d)",
    () -> callInterpolate(MPI_OK, X_rows(null, new double[]{}{0,0}, new double[]{}{1,1}), new double[]{}{0.0, 0.0, 2.0}),
    DimensionMismatchException.class);
// Z314: d≥1 & |X|≥1 & |X[0]|≠d → DimensionMismatch
expectThrow("Z314 |X[0]|!=d",
    () -> {
        Object mpi3 = newMPI_auto(3, 10, "OK", true);
        callInterpolate(mpi3,
            X_rows(new double[]{}{0,0}, new double[]{}{1,1},
new double[]{}{2,2}, new double[]{}{3,3}),
            new double[]{}{0,2,4,6});
    },
    DimensionMismatchException.class);
// Z315: |X| < d+1 → NumberIsTooSmall
expectThrow("Z315 n<d+1",
    () -> {
        Object mpi4 = newMPI_auto(4, 10, "OK", true);
        callInterpolate(mpi4,
            X_rows(new double[]{}{0,0,0,0}, new double[]{}{1,1,1,1}, new double[]{}{2,2,2,2}, new double[]{}{3,3,3,3}),
            new double[]{}{0,1,2,3}); // n=4, d=4 → n<d+1
    }

```

```

        },
        NumberIsTooSmallException.class);
// Z316: non-finite in X or Y → NotFiniteNumber
expectThrow("Z316 non-finite X/Y",
    () -> {
        Object mpi1 = newMPI_auto(1, 8, "OK", true);
        callInterpolate(mpi1,
            X_rows(new double[] {0}, new double[] {Double
.NaN}, new double[] {2}),
            new double[] {0.0, 1.0, 2.0});
    },
    NotFiniteNumberException.class);
// Z317: Interpolate: OK → None
headline("Z317 interpolate OK");
Object fOK;
{
    Object mpi2 = newMPI_auto(2, 16, "OK", true);
    double[][] X = X_rows(new double[] {0,0}, new double[] {1,0},
new double[] {0,1}, new double[] {1,1}, new double[] {2,1});
    double[] Y = new double[] {0.0, 1.0, 2.0, 3.0, 4.0};
    fOK = callInterpolate(mpi2, X, Y);
    System.out.println("Z317 interpolate success (n=5,d=2)");
}
// Z318: Value: p=null → NullArgument
expectThrow("Z318 value p=null",
    () -> callValue(fOK, null),
    NullArgumentException.class);
// Z319: Value: |p| ≠ d → DimensionMismatch
expectThrow("Z319 value |p|≠d",
    () -> callValue(fOK, new double[] {1.0}), // d=2 時
    DimensionMismatchException.class);
// Z320: Value: OK (|p|=d & finite) → None
headline("Z320 value OK");
{
    double val = callValue(fOK, new double[] {0.5, 0.25});
    System.out.printf("Z320 value -> %s\n", fmt(val));
}
System.out.println("\n[Done] MicrosphereProjectionInterpolator
Z301-Z320 executed.");
}
}
final class MPI_VT_Runner {
    public static void main(String[] args) throws Exception {

```

```

new MPI_VT().Vtest();
// new MPI_FT().Ftest();
// new MPI_Z3().Z3test();

}
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 93.8%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
MicrosphereProjectionInterpolator.java	20.2 %	123	485	608
> MPI_VT	52.4 %	108	98	206
MicrosphereProjectionInterpolator	93.8 %	15	1	16
MicrosphereProjectionInterpolator(ir)	50.0 %	1	1	2
> interpolate(double[][], double[])	100.0 %	14	0	14
MicrosphereProjectionInterpolator(ir)		0	0	0
> MPI_VT_Runner		0	0	0

3. V-Method testing instruction coverage: 94.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
MicrosphereProjectionInterpolator.java	30.5 %	1,577	3,587	5,164
> MPI_VT_Runner	62.5 %	5	3	8
> MPI_VT	75.3 %	1,432	469	1,901
MicrosphereProjectionInterpolator	94.8 %	110	6	116
MicrosphereProjectionInterpolator(ir)	76.0 %	19	6	25
MicrosphereProjectionInterpolator(ir)	100.0 %	18	0	18
> interpolate(double[][], double[])	100.0 %	73	0	73

4. Fuzz testing branch coverage: 62.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
MicrosphereProjectionInterpolator.java	22.7 %	138	470	608
MicrosphereProjectionInterpolator	62.5 %	10	6	16
MicrosphereProjectionInterpolator(ir)	50.0 %	1	1	2
> interpolate(double[][], double[])	64.3 %	9	5	14
MicrosphereProjectionInterpolator(ir)		0	0	0
> MPI_FT	64.0 %	128	72	200
> MPI_VT_Runner		0	0	0

5. Fuzz testing instruction coverage: 73.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
MicrosphereProjectionInterpolator.java	23.9 %	1,234	3,930	5,164
> MPI_VT_Runner	62.5 %	5	3	8
MicrosphereProjectionInterpolator	73.3 %	85	31	116
> interpolate(double[][], double[])	65.8 %	48	25	73
MicrosphereProjectionInterpolator(ir)	76.0 %	19	6	25
MicrosphereProjectionInterpolator(ir)	100.0 %	18	0	18
> MPI_FT	79.9 %	1,114	281	1,395

6. Z3 branch coverage: 81.2%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
MicrosphereProjectionInterpolator.java	16.4 %	100	508	608
> MPI_Z3	46.8 %	87	99	186
MicrosphereProjectionInterpolator	81.2 %	13	3	16
MicrosphereProjectionInterpolator(ir)	50.0 %	1	1	2
> interpolate(double[][], double[])	85.7 %	12	2	14
MicrosphereProjectionInterpolator(ir)		0	0	0
> MPI_VT_Runner		0	0	0

7. Z3 instruction coverage: 92.2%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼  MicrosphereProjectionInterpolator.java		25.8 %	1,334	3,830	5,164
>  MPI_VT_Runner		62.5 %	5	3	8
>  MPI_Z3		69.5 %	1,192	522	1,714
▼  MicrosphereProjectionInterpolator		92.2 %	107	9	116
 MicrosphereProjectionInterpolator(Ir		76.0 %	19	6	25
>  interpolate(double[], double[])		95.9 %	70	3	73
 MicrosphereProjectionInterpolator(ir		100.0 %	18	0	18